

National University of Computer and Emerging Sciences



Assignment No. 3

Data Structures

Binary Trees

CS2001

Fall 2023

Deadline: November 16, 2023 by 4:00 P.M

Submission Instructions:

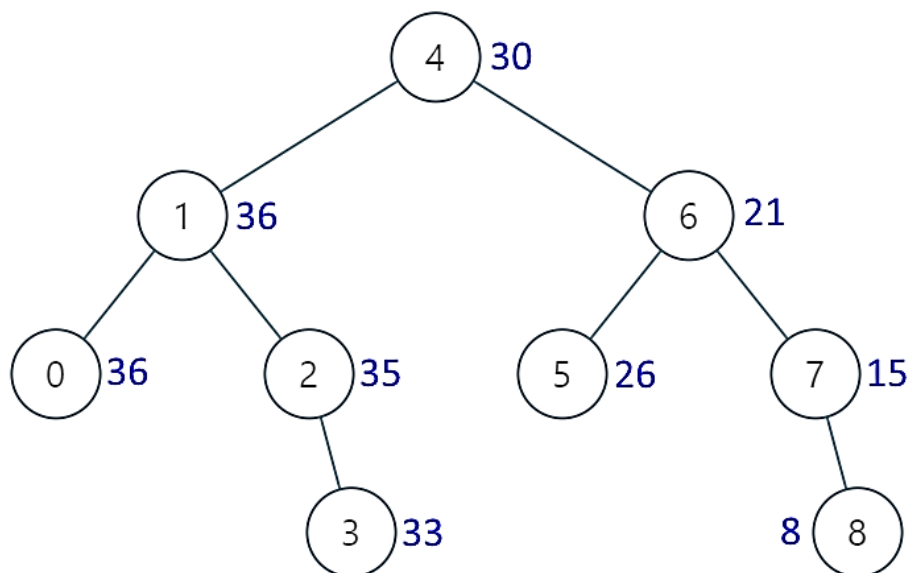
- All problems must be solved by following the order and submitted as one .zip file.
- Copy all the header (.h) and (.cpp) files in a folder with the naming convention given below containing separate subfolders for each question.
- Parent folder should be named as **DS_ASS03_XXF_YYYY**, where XX represents the batch and YYYY represents the roll number. Compress this folder as **DS_ASS03_XXF_YYYY.zip** and submit this one file.
- Solution of each question should be in a subfolder, named according to the questions.
- Also submit a **.doc file** containing all the codes along with their **screenshots**.
- Any submission not following the submission instructions will not be evaluated.
- This is an individual assignment.
- **Design proper classes rather than global functions.**
- **Plagiarism is strictly prohibited.**

Question 01: Algebraic Expression

Draw a binary tree for the expression $A*B-(C+D)*(P/Q)$.

Question 02: Greater Tree

Given the root of a Binary Search Tree (BST), convert it to a Greater Tree such that every value of the original BST is changed to the original value plus the sum of all nodes greater than the current in BST. Implement methods properly.

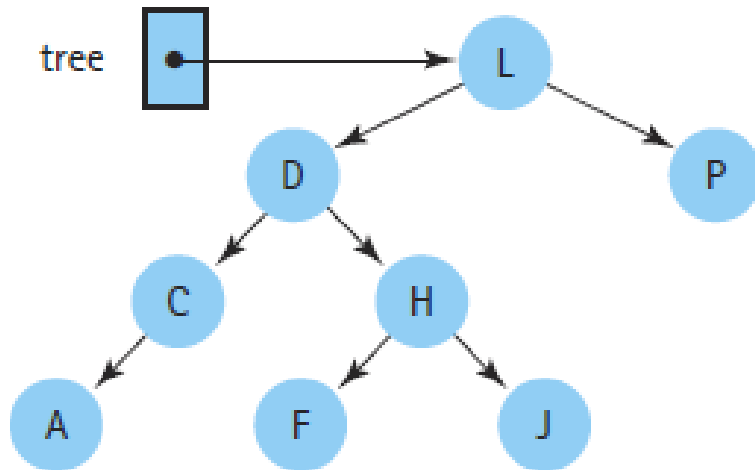


Question 03: Deletion

Delete the nodes from the below BST in the following order:

J, C, L, D, A

When deleting a node with two children, use the immediate predecessor. Handle the other two cases as well and implement properly.

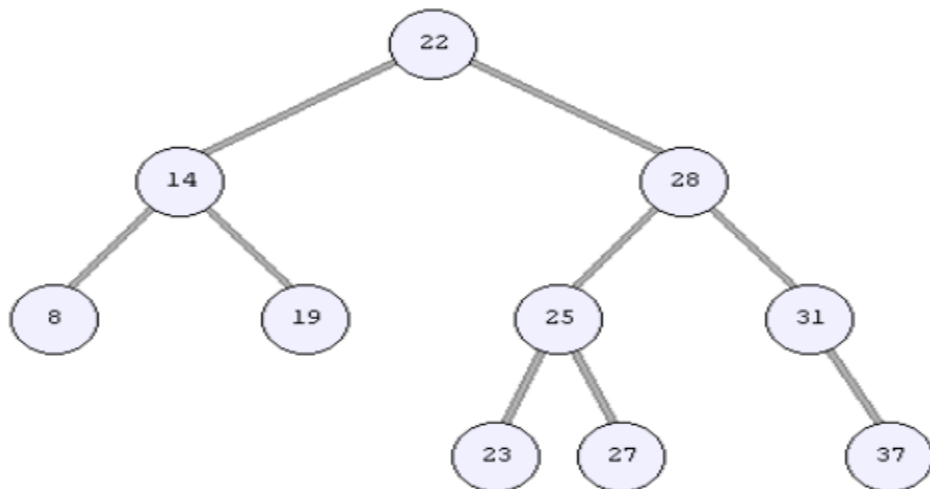


Question 04: Sub AVL

Design a function that takes two values as input from the user and returns true if the subtree of a given AVL tree, rooted between those two values, is itself an AVL tree. Otherwise, it returns false.

Question 05: Draw AVL Tree

Starting with the AVL tree below, insert the following values: 26, 34, 40, 29, 33, 32. Show the resulting tree after each insertion.



Question 06: Student Enrollment System

You are developing a student enrollment system for FAST University. Each student is identified by their unique student ID, and you intend to employ an AVL tree to store the students' information. The AVL tree will be organized based on student IDs. These student IDs are both unique and meaningful. For example, the first digit represents the student's department, to which they belong, while the remaining digits represent the student's unique ID.

Here are some sample student IDs:

Student ID: 1XXX | Department: CS

Student ID: 2XXX | Department: SE

Student ID: 3XXX | Department: EE

Student ID: 4XXX | Department: S&H

Student ID: 5XXX | Department: FSM

Your task is to store each student's ID, name, and department in a node and construct an AVL tree based on student IDs. The program should be menu-driven, allowing administrators of FAST University to change a student's department (the first digit of student's ID will automatically change accordingly). Your program should efficiently manage the tree structure after updating a student's department and ID.

Check your program by entering following student IDs

1020, 3045, 1109, 5333, 2320, 2980, 4011, 3041, 4049

Question 07: Delete Kth largest

Take n inputs from the user and construct an AVL tree using those values. Next, take an integer k from the user and delete the k th largest value node from the constructed AVL tree.

Question 08: Most Frequent Songs

You have recently joined Spotify Technology S.A. as a software engineer, and your task is to design an algorithm and develop a menu-driven program that efficiently stores and manages the most frequently played songs in an AVL tree. Each node in the **Heap** tree should contain the song name and its frequency, and the tree should be constructed based on the song frequency. Additionally, the program should allow users to play songs. Once a song is played, its frequency will increase by 1, ensuring the tree remains balanced.

If two or more songs have the same frequencies, then add songs with the same frequency next to each other by creating a linked list of nodes in the tree.

To get you started, you are provided with the following initial song frequencies:

Song_A: Frequency = 1

Song_B: Frequency = 5

Song_C: Frequency = 9

Song_D: Frequency = 2

Song_E: Frequency = 4

Song_F: Frequency = 6

Song_G: Frequency = 8

Song_H: Frequency = 3

Song_I: Frequency = 7

Song_J: Frequency = 9

Song_K: Frequency = 5

Question 09: Patient Admission System

You are building a program for a hospital to manage its patient waiting system. The hospital has limited beds, and they want to allot beds in a way that prioritizes those with the most severe conditions. To achieve this, use a Max-Heap data structure to keep track of patients' conditions and prioritize them on their condition severity (suppose each patient has a unique severity condition).

Design a menu-driven program that uses a Max-Heap to manage the hospital's patient waiting system. You should create a Patient class with the following attributes:

```
struct Patient {  
    string name;  
    int condition; // A higher value indicates a more severe condition.  
  
    Patient(string name, int condition) {  
        this->name = name;  
        this->condition = condition;  
    }  
};
```

You are required to implement the following functions in your menu-driven program:

1. **void** add_patient(**Patient** patient): This function should add a new patient to the waiting system by inserting them into the Max-Heap based on their severity condition.
2. **Patient** allot_bed(): This function should remove (allot bed) and return the patient with the highest severity condition from the waiting system.
3. **int** num_patients(): This function should return the current number of patients in the admission system.